Cálculo Numérico

Hiroshi Nakamura

ifms.edu.br

22 de março de 2023

Indice I

1. Erros
   - IEEE 754
   - Convertendo o valor decimal para IEEE 754
   - Convertendo o valor IEEE 754 para decimal
   - IEEE 754 - Valores especiais
   - Erros Absoluto e Relativo

Ponto flutuante precisão simples = C -> float

Sinal	Expoente								Mantissa																					
+/-	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	2^{-7}	2^{-8}	2^{-9}	2^{-10}	2^{-11}	2^{-12}	2^{-13}	2^{-14}	2^{-15}	2^{-16}	2^{-17}	2^{-18}	2^{-19}	2^{-20}	2^{-21}	2^{-22}	2^{-23}
0	1	0	0	0	0	0	1	0	1	0	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
31=MSB	30							23	22																					0=LSB

Tabela: 32 bits = 1 bit sinal + 8 bits expoente + 23 bits mantissa

O número acima apresentado é o $13{,}375_{10}$ que no formato IEEE 754 é $01000001010101100000000000000000_2$, modo o qual é armazenado em memória em uma variável **float** em linguagem C.

ieee754simples.c

saru@computer4en6: ~/IFMS/2021_1/calculonumerico/20210322aula02

Arquivo Editar Ver Pesquisar Terminal Ajuda

```c
#include <stdio.h> //printf()
#include <math.h> // modff()

int
main()
  {
  float simples = 13.375;
  float inteiro;
  float fracao;
  float mantissa;
  int   expoente;

  printf("Padrao = %f\n", simples);
  fracao = modff(simples, &inteiro);
  printf("Parte inteira = %f e fracionaria = %f\n\n", inteiro, fracao);
  printf("Cientifica = %e\n", simples);
  return(0);
  }
```

Saída do programa **ieee754simples**

Programa **ieee754simples** executado

```
Padrao = 13.375000
Parte inteira = 13.000000 e fracionaria = 0.375000
Cientifica = 1.337500e+01
```

Compilando com GCC e observando o padrão IEEE 754 com GDB - 1

Compilando o **ieee754simples.c** com suporte a depuração

```
gcc -ggdb -o ieee754simples ieee754simples.c
```

Executando o **ieee754simples** no debugger

```
gdb ./ieee754simples
```

Inserindo um ponto de parada na linha 17

```
(gdb) break 17
```

Executando o programa que vai ser parado na linha 17

```
(gdb) run
```

Compilando com GCC e observando o padrão IEEE 754 com GDB - 2

Examinando a memória da variável **simples** e vendo seu valor em binário no formato IEEE 754

```
(gdb) x/tw &simples
0x7fffffffdd0c:01000001010101100000000000000000
```

Para a compreensão de como $13{,}375_{10}$ se tranformou em $01000001010101100000000000000000_2$ no formato IEEE 754 precisaremos converter a parte inteira e a fracionária do número em formato decimal.

Convertendo o valor decimal para IEEE 754 - 1

-

1º passo - converter a parte inteira do **decimal** para **binário** 13/2 = 6 resto 1
6/2 = 3 resto 0
3/2 = 1 resto 1
Parte INTEIRA em BINÁRIO = 1101_2

2º passo - converter a parte fracionária do **decimal** para **binário**

0,375 * 2 = 0,75
0,75 * 2 = 1,5 (Se a parte a direita da vírgula não for zero, ignorar o 1 para a próxima conta)
0,5 * 2 = 1,0
Parte FRACIONÁRIA em BINÁRIO = 011_2

Resultado do valor decimal convertido para binário

Resultado em BINÁRIO = $1101,011_2$

Convertendo o valor decimal para IEEE 754 - 2

Converter para o resultado o formato exigido

$-1^{sinal} * (1,m) * 2^{n}$

3º passo - movendo o ponto flutuante(vírgula) n casas a esquerda para adequar ao padrão

1101,011

RESULTADO = $+1,101011 * 2^{3}$

4º passo - Somar o valor de n a 127 para achar o EXPOENTE em decimal

EXPOENTE em DECIMAL = $127 + 3 = 130_{10}$

5º passo - Converter de decimal para binário deixando o EXPOENTE no formato IEEE 754

$130_{10} = 10000010_{2}$

Convertendo o valor decimal para IEEE 754 - 3

6° passo - Removendo o **1**, após a movimentaçãdo ponto flutuante para achar a MANTISSA

1,101011
RESULTADO = 10101100000000000000000

7° passo - FINAL - Unir o SINAL ao EXPOENTE e a MANTISSA

01000001010101100000000000000000

Convertendo o valor IEEE 754 para decimal - 1

1º passo - Identificar o SINAL através do **MSB**

01000001**1**01011000000000000000000

Como o MSB é **0** então nosso número decimal será + (positivo).

2º passo - Converter a MANTISSA de binário para decimal

$10000010_2 = 130_{10}$

3º passo - Retirar 127 do valor decimal para o valor n

n = 130 - 127 = 3

4º passo - Inserir **1,** a esqueda do valor da MANTISSA

1,10101100000000000000000

1,**101011**

Convertendo o valor IEEE 754 para decimal - 2

5° passo - mover o ponto flutuante(vírgula) n casas a direita para voltar ao binário fracionário

1,101011
n = 3
BINÁRIO FRACIONÁRIO = $1101,011_2$

6° passo - converter a parte inteira do BINÁRIO para DECIMAL

$1101_2 = 1 * 2^3 + 1 * 2^2 + 0 * 2^1 + 1 * 2^0 = 13_{10}$

7° passo - converter a parte fracionária do BINÁRIO para DECIMAL

$011_2 = 0 * 2^{-1} + 1 * 2^{-2} + 1 * 2^{-3} = 0{,}375_{10}$

8° passo - FINAL - Unir as partes INTEIRA e FRACIONÁRIA do DECIMAL

DECIMAL = 13 + 0,375 = $\mathbf{13{,}375}_{10}$

IEEE 754 - Desnormalizado e NaN

Sinal	Expoente								Mantissa																			
0 ou 1	0	0	0	0	0	0	0	0	1	0	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Tabela: Número desnormalizado = 0 ou 1 + tudo 0 + 0 ao máximo

Número desnormalizado

O número não está no padrão de formatação IEEE 754.

Sinal	Expoente								Mantissa																			
0 ou 1	1	1	1	1	1	1	1	1	1	0	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Tabela: NaN = 0 ou 1 + tudo 1 + diferente de 0

NaN

NaN - Not a Number - Valor que não pode ser representado numericamente.

IEEE 754 - Infinito Positivo e Infinito Negativo

Sinal	Expoente								Mantissa																			
0	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Tabela: $+\infty$ = 0 + tudo 1 + tudo 0

$+\infty$

$+\infty$ ou **inf** - Infinito positivo.

Sinal	Expoente								Mantissa																			
1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Tabela: -∞ = 1 + tudo 1 + tudo 0

-∞

-∞ ou **-inf** - Infinito negativo.

ieee754especiais

saru@computer4en6: ~/sul/aulas/calculonumerico

Arquivo Editar Ver Pesquisar Terminal Ajuda

```
#include <stdio.h> // printf()
#include <math.h>  // sqrt()

int
main()
  {
  double x = 1.0;
  double d;

  d = x/0.0;
  printf("%f/0.0 = %f\n", x, d);

  d = -x/0.0;
  printf("%f/0.0 = %f\n\n", -x, d);

  d = sqrt(-x);
  printf("sqrt(%f) = %f\n" , -x, d);

  return(0);
  }
```

Compilando com GCC o ieee754especiais.c

Compilando o **ieee754especiais.c**

```
gcc -ggdb -o ieee754especiais ieee754especiais.c -lm
```

Programa **ieee754especiais.c** executado

```
1.000000/0.0 = inf
-1.000000/0.0 = -inf
sqrt(-1.000000) = -nan
```

Truncamento e Arredondamento

Truncamento

Truncar um número é limitar a quantidade de dígitos na sua parte fracionária.
1.75 truncado 1 um dígito fica 1.7
1.74 truncado 1 um dígito fica 1.7

Arredondamento

O Arredondamento acontece quando diminuímos a precisão a ser mostrada pelo computador. Quando o valor do dígito fracionário é maior ou igual a metade, o valor é arredondado para cima, no caso contrário é arredondado para baixo.
1.750000 arredondado fica 1.8
1.749999 arredondado fica 1.7

arredondamento.c

saru@computer4en6: ~/sul/aulas/calculonumerico

Arquivo Editar Ver Pesquisar Terminal Ajuda

```c
#include <stdio.h> //printf()
#include <math.h>

int
main()
  {
  float x = 1.750000;
  float y = 1.749999;

  printf("Arredondamento de %f = %.1f\n", x, x);

  printf("Arredondamento de %f = %.1f\n", y, y);

  return(0);
  }
```

Compilando com GCC o arredondamento.c

Compilando o **arredondamento.c**

```
gcc -ggdb -o arredondamento arredondamento.c -lm
```

Programa **arredondamento.c** executado

```
Arredondamento de 1.750000 = 1.8
Arredondamento de 1.749999 = 1.7
```

Conversão de bases

Erro de conversão de base

Na conversão entre decimal e binário pode ocorrer erro na sua representação computacional.

$0{,}2_{10}$ na representação binária de um computador acaba ocorrendo em um dízima periódica 00110011...

E nesse processo se perde a precisão.

0.200000 com a precisão de 9 digítos = 0.200000003

erroconversaobase.c

saru@computer4en6: ~/sul/aulas/calculonumerico

Arquivo Editar Ver Pesquisar Terminal Ajuda

```c
#include <stdio.h> //printf()

int
main()
  {
  float x = 0.2;

  printf("Padrao = %f\n", x);
  printf("Com precisao de 9 digitos = %.9f\n", x);
  return(0);
  }
```

Compilando com GCC o erroconversaobase.c

Compilando o **erroconversaobase.c**

```
gcc -ggdb -o erroconversaobase erroconversaobase.c
```

Programa **erroconversaobase.c** executado

```
Padrao = 0.200000
Com precisao de 9 digitos = 0.200000003
```

Erro Absoluto e Erro Relativo

Erro

Durante o truncamento, arredondamento e também na conversão de bases podem acontecer erros.

Erro Absoluto

O Erro absoluto é o valor real do erro. Erro absoluto tem a seguinte fórmula:
$E_{abs} = |x + \overline{x}|$, onde x é o valor exato e $\overline{x}$ é o valor aproximado.

Erro Relativo

O Erro relativo é o valor percentual do erro. Erro relativo tem a seguinte fórmula:

$$E_{rel} = \frac{|x + \overline{x}|}{x}$$

erroabsolutorelativo.c

saru@computer4en6: ~/sul/aulas/calculonumerico

Arquivo Editar Ver Pesquisar Terminal Ajuda

```c
#include <stdio.h> // printf()
#include <math.h> // fabs()

int
main()
  {
  double x_exato = 100.0;
  double x_aproximado = 99.99;
  double erro_absoluto;
  double erro_relativo;

  erro_absoluto = fabs( x_exato - x_aproximado );
  printf("Erro Absoluto = %f\n", erro_absoluto);

  erro_relativo = ( (fabs( x_exato - x_aproximado))/x_exato );
  printf("Erro Relativo = %f em porcentagem %f%%\n", erro_relativo, erro_relativo * 100.0);
  printf("Erro Relativo = %.4f em porcentagem %.2f%%\n", erro_relativo, erro_relativo * 100.0);
  }
```

Compilando com GCC o erroabsolutorelativo.c

Compilando o **erroabsolutorelativo.c**

```
gcc -o erroabsolutorelativo erroabsolutorelativo.c -lm
```

Programa **erroabsolutorelativo.c** executado

```
Erro Absoluto = 0.010000
Erro Relativo = 0.000100 em porcentagem 0.010000%
Erro Relativo = 0.0001 em porcentagem 0.01%
```